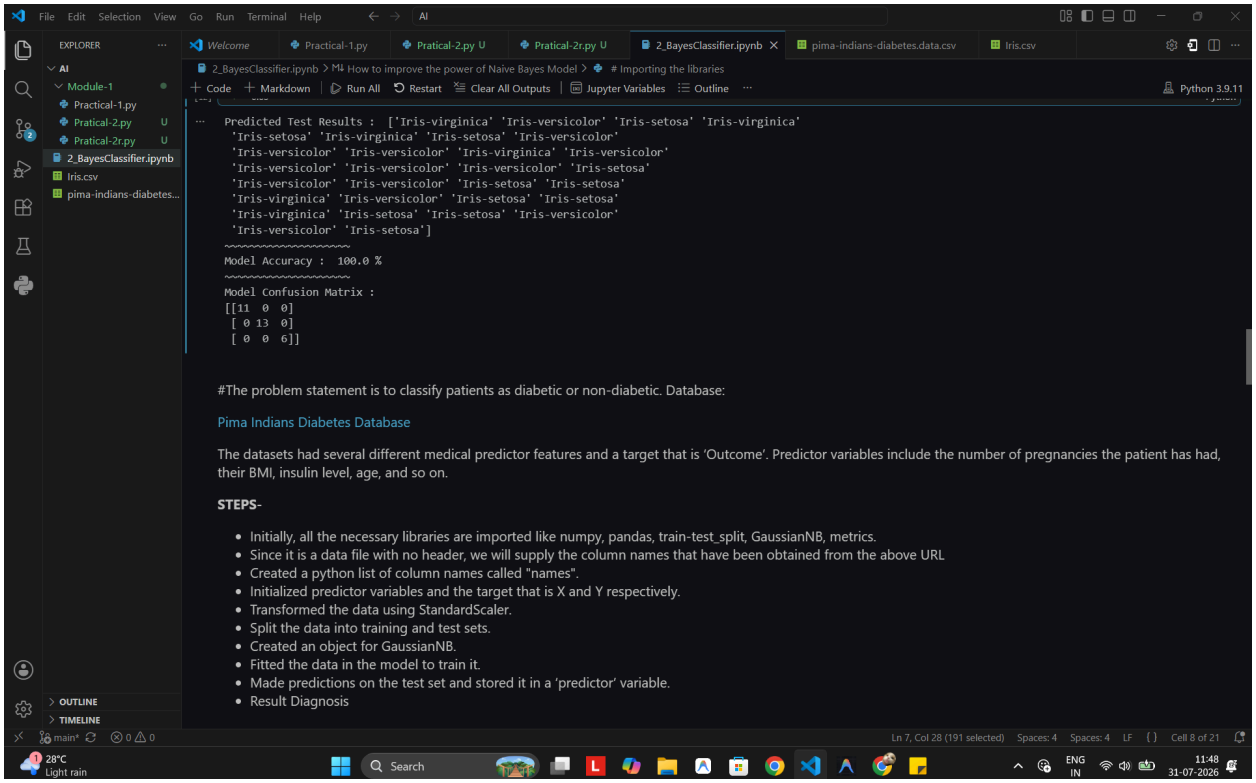
Aim: Naive Bayes' Classifier

Implement the Naive Bayes' algorithm for classification.Train a Naive Bayes' model using a given dataset and calculate class Probabilities. Evaluate the accuracy of the model on test data and analyze the results.

```
# Importing the libraries
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

# Importing the dataset
dataset = pd.read_csv('iris.csv')
X = dataset.iloc[:, [0, 3]].values
y = dataset.iloc[:, -1].values

# Splitting the dataset into the Training set and Test set
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size =
0.20, random_state = 0)

# Feature Scaling
#Since our dataset containing character variables
#we have to encode it using LabelEncoder
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)

# Training the Naive Bayes model on the Training set
from sklearn.naive_bayes import GaussianNB
classifier = GaussianNB()
classifier.fit(X_train, y_train)

# Predicting the Test set results
y_pred = classifier.predict(X_test)
print("Predicted Test Results : ",y_pred)
print("~"*20)

# Making the Confusion Matrix
from sklearn.metrics import confusion_matrix, accuracy_score
```

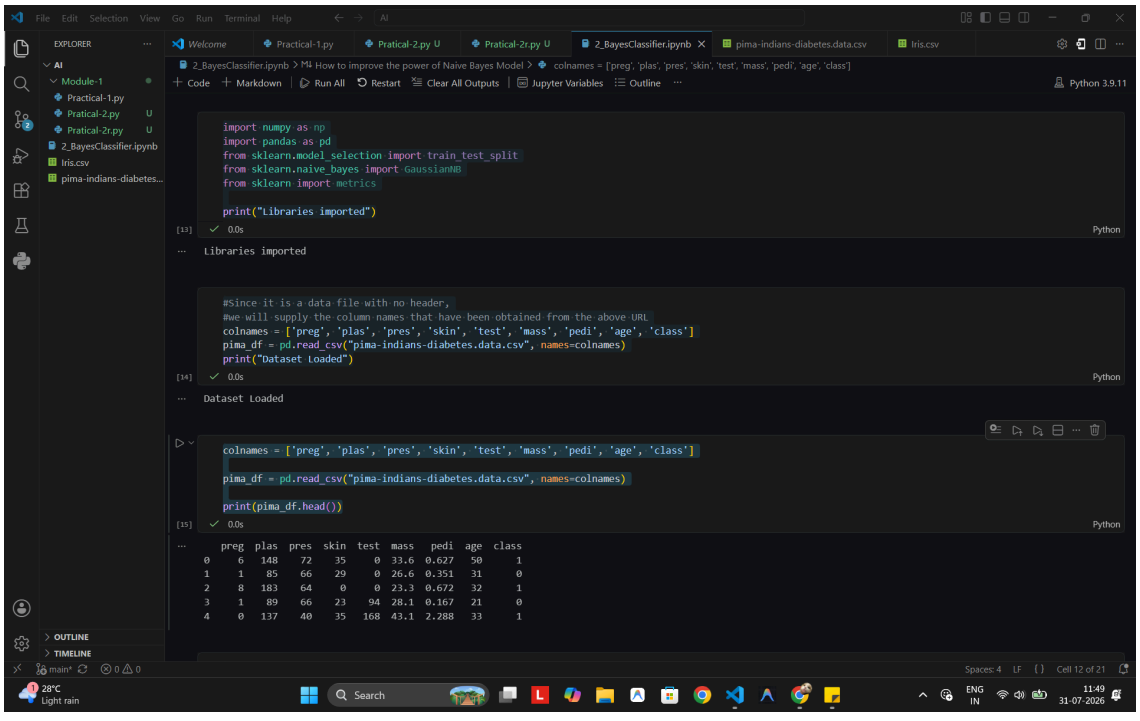
```
ac = accuracy_score(y_test,y_pred)
print("Model Accuracy : ",ac*100,"%")
print("~"*20)
cm = confusion_matrix(y_test, y_pred)
print("Model Confusion Matrix : ")
print(cm)
```



```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn import metrics

print("Libraries imported")
```

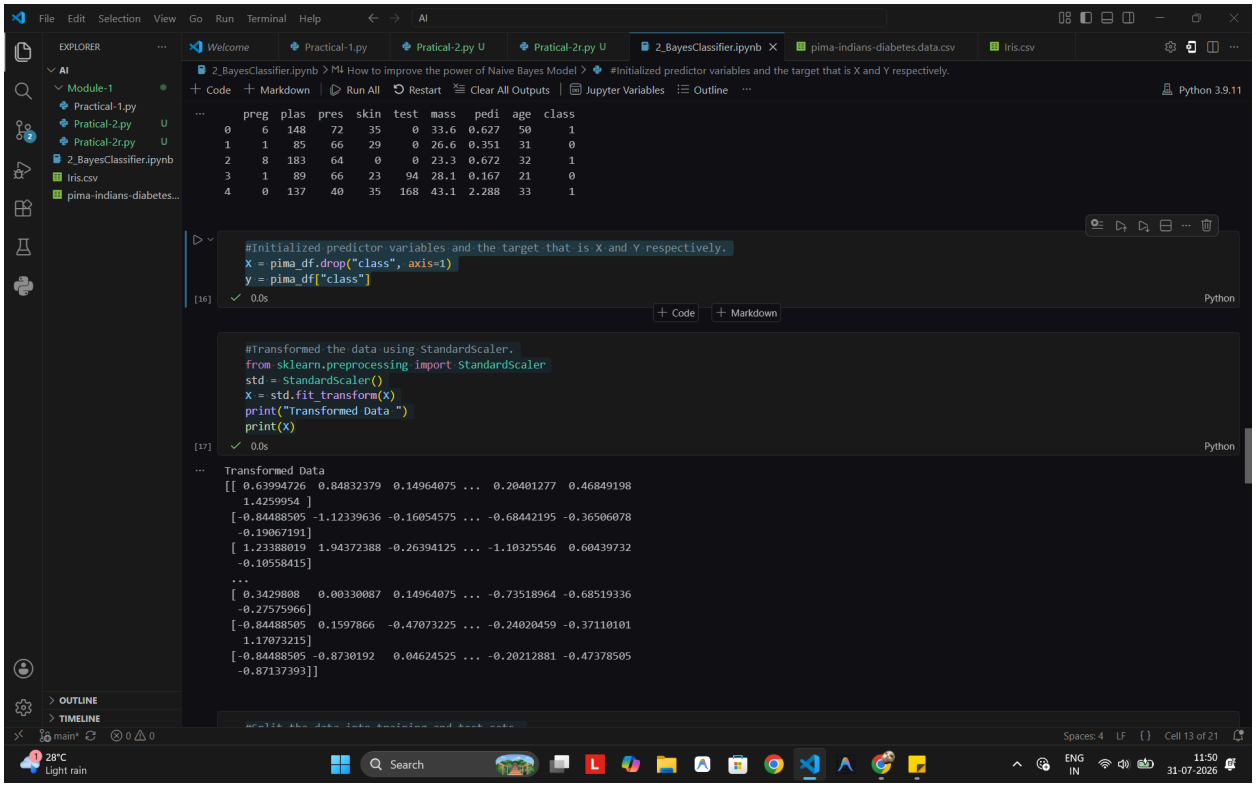
```
#Since it is a data file with no header,
#we will supply the column names that have been obtained from the above
URL
colnames = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age',
'class']
pima_df = pd.read_csv("pima-indians-diabetes.data.csv", names=colnames)
print("Dataset Loaded")
```

```
colnames = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age',
'class']

pima_df = pd.read_csv("pima-indians-diabetes.data.csv", names=colnames)

print(pima_df.head())
```



```
#Initialized predictor variables and the target that is X and Y
respectively.
X = pima_df.drop("class", axis=1)
y = pima_df["class"]
```

```
#Transformed the data using StandardScaler.
from sklearn.preprocessing import StandardScaler
std = StandardScaler()
X = std.fit_transform(X)
print("Transformed Data ")
print(X)
```

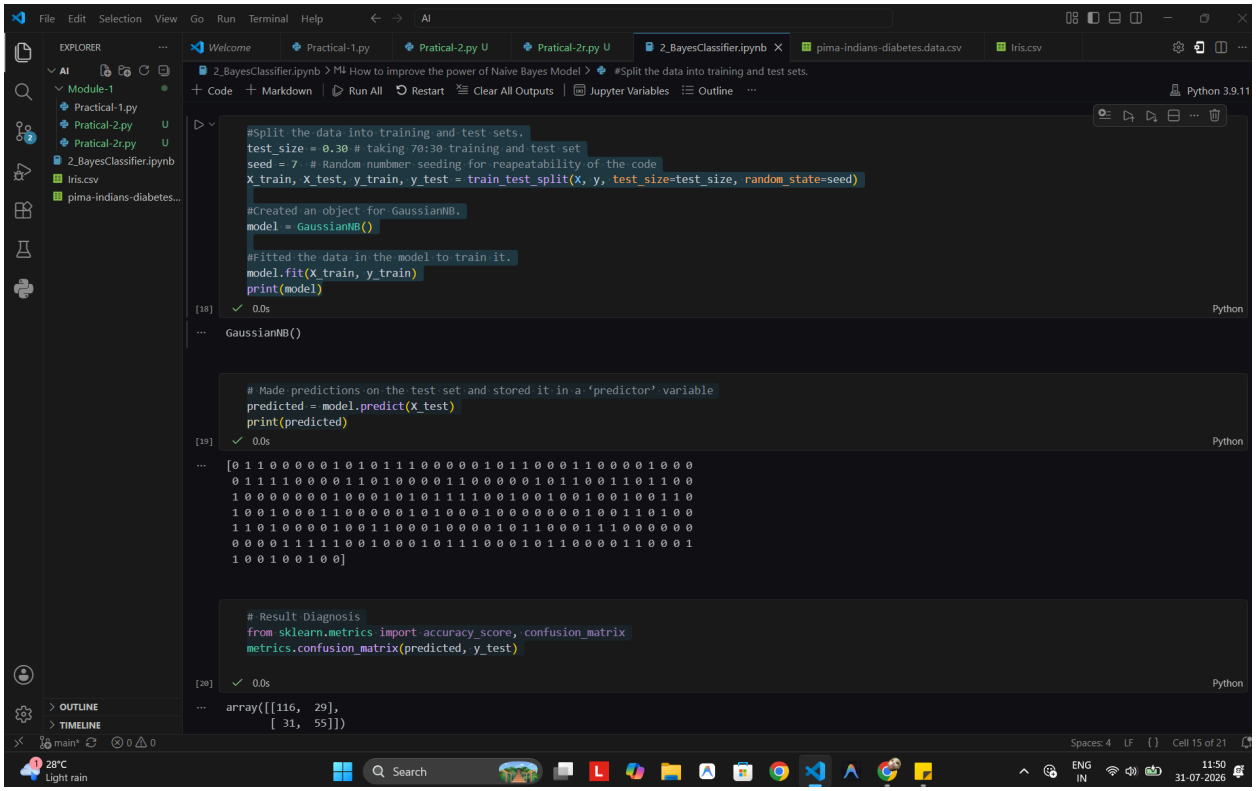


```
#Split the data into training and test sets.
test_size = 0.30 # taking 70:30 training and test set
seed = 7  # Random numbmer seeding for reapeatability of the code
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=test_size, random_state=seed)

#Created an object for GaussianNB.
model = GaussianNB()

#Fitted the data in the model to train it.
model.fit(X_train, y_train)
print(model)
```

```
# Made predictions on the test set and stored it in a 'predictor' variable
predicted = model.predict(X_test)
print(predicted)
```

```
# Result Diagnosis
from sklearn.metrics import accuracy_score, confusion_matrix
metrics.confusion_matrix(predicted, y_test)
```

Sheth L.U.J & Sir M.V College
Subject:AI



```
#Split the data into training and test sets.
test_size = 0.30 # taking 70:30 training and test set
seed = 7  # Random numbmer seeding for reapeatability of the code
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=test_size, random_state=seed)

#Created an object for GaussianNB.
model = GaussianNB()

#Fitted the data in the model to train it.
model.fit(X_train, y_train)
print(model)
```

```
GaussianNB()
```

```
# Made predictions on the test set and stored it in a 'predictor' variable
predicted = model.predict(X_test)
print(predicted)
```

```
[0 1 1 0 0 0 0 0 1 0 1 0 1 1 1 0 0 0 0 0 1 0 1 1 0 0 0 1 1 0 0 0 0 1 0 0 0
 0 1 1 1 1 0 0 0 0 1 1 0 1 0 0 0 0 0 1 1 0 0 0 0 0 1 0 1 1 0 0 1 1 0 1 1 0 0
 1 0 0 0 0 0 0 0 1 0 0 0 1 0 1 0 1 1 1 1 0 0 1 0 0 1 0 0 1 0 0 1 0 0 1 1 1 0
 1 0 0 1 0 0 0 1 1 0 0 0 0 0 1 0 1 0 0 0 1 0 1 0 0 0 0 0 0 0 1 0 0 1 1 0 1 0 0
 1 1 0 1 0 0 0 0 1 0 0 1 1 0 0 0 1 0 0 0 0 1 0 1 1 1 0 0 0 1 1 1 0 0 0 0 0 0
 0 0 0 0 1 1 1 1 1 0 0 1 0 0 0 1 0 1 1 1 0 0 0 1 0 1 1 0 0 0 0 1 1 0 0 0 1
 1 0 0 1 0 0 1 0 0]
```

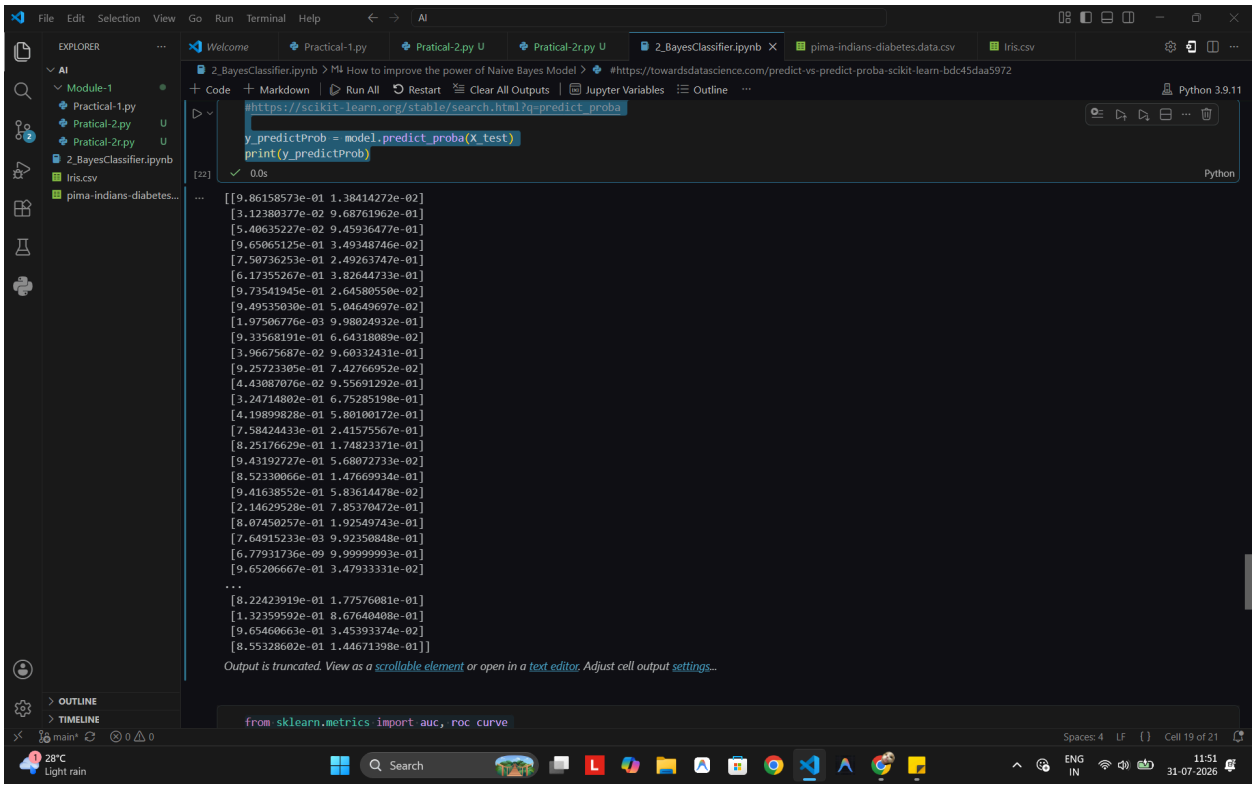
```
# Result Diagnosis
from sklearn.metrics import accuracy_score, confusion_matrix
metrics.confusion_matrix(predicted, y_test)
```

```
array([[116,  29],
       [ 31,  55]])
```

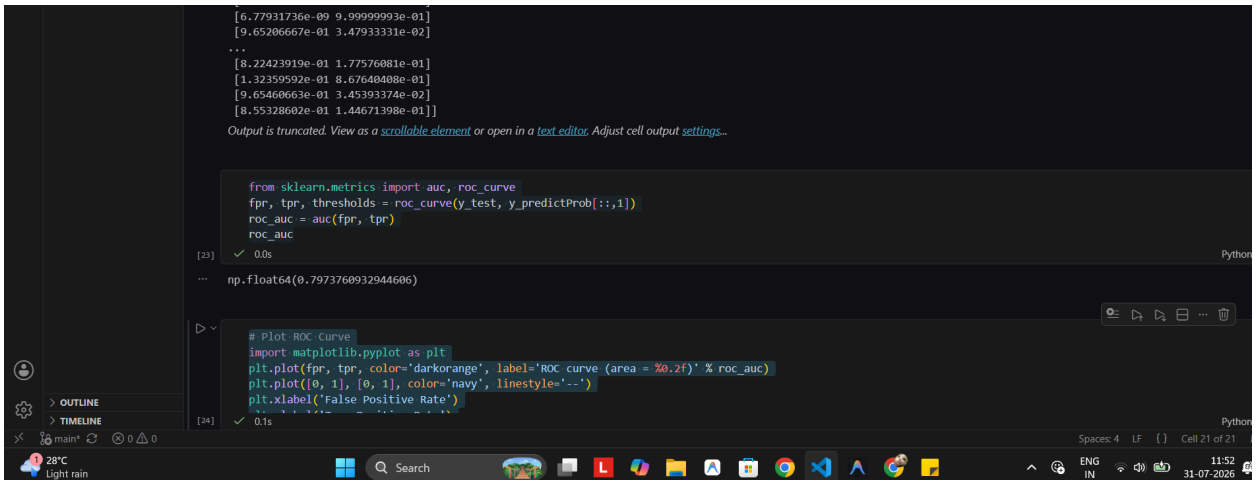
Carol Pillai
T104

```
#https://scikit-learn.org/stable/modules/model_evaluation.html
model_score = model.score(X_test, y_test)
model_score
```

```
#https://towardsdatascience.com/predict-vs-predict-proba-scikit-learn-bdc4
5daa5972
#https://scikit-learn.org/stable/search.html?q=predict_proba

y_predictProb = model.predict_proba(X_test)
print(y_predictProb)
```



```
from sklearn.metrics import auc, roc_curve
fpr, tpr, thresholds = roc_curve(y_test, y_predictProb[::,1])
roc_auc = auc(fpr, tpr)
roc_auc
```



```
# Plot ROC Curve
import matplotlib.pyplot as plt
plt.plot(fpr, tpr, color='darkorange', label='ROC curve (area = %0.2f)' %
roc_auc)
plt.plot([0, 1], [0, 1], color='navy', linestyle='--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver operating characteristic')
plt.legend(loc="lower right")
```